Functions Notes

16 July 2025 12:52

Definition:

A **function** is a named block of memory that stores a set of instructions to perform a specific task. Once a function is created, it can be reused any number of times without rewriting the code.

Why Do We Need Functions? *(Important)*

- To reduce the number of instructions in a program.
- To improve the efficiency and readability of code.
- To avoid code repetition.
- To reuse the same block of code multiple times.

Types of Functions:

- **Inbuilt Functions**
- **User-defined Functions**

Inbuilt Functions:

Classifications of Inbuilt Functions:

1.Utility functions
2.String Inbuilt Functions
3.List Inbuilt Functions
4.Tuple Inbuilt Functions
5.set Inbuilt Functions
6.Dictionary Inbuilt Functions

Utility functions:

These are predefined functions provided by Python to perform common tasks.
Examples: print(), len(), sum(), type().

2.String Inbuilt Functions:

1. len()
Returns the total number of characters in the string, including spaces and symbols. It helps in knowing the length of a string.
Syntax:variablename.len(variablename)
Example:

```
s = "hello"
print(len(s))  # Output: 5
```

2. upper()
Converts all the characters in a string to uppercase letters. Useful for formatting and comparison.
Syntax:variablename.upper()
Example:

```
s = "hello"
print(s.upper())  # Output: "HELLO"
```

3. lower()
Changes all characters in the string to lowercase. Helps standardize text input.
Syntax:variablename.lower()

Example:

```
s = "HELLO"
print(s.lower())  # Output: "hello"
```

4. capitalize()

Converts the first character to uppercase and makes the rest lowercase. Commonly used in names or titles.
Syntax:variablename.capitalize()

Example:
s = "python"
print(s.capitalize()) # Output: "Python"

5. title()
Converts the first letter of **each word** to uppercase. Used for headings or proper formatting.
Syntax:variablename.title()
Example:

s = "python is fun"
print(s.title()) # Output: "Python Is Fun"

6. strip()
Removes spaces from both ends of the string. Helps in cleaning user input.
Syntax:variablename.strip()
Example:
s = " hello "
print(s.strip()) # Output: "hello"

7. lstrip()
Removes spaces from the **left** side only. Useful when formatting text.
Syntax:variablename.lstrip()
Example:
s = "   hello"
print(s.lstrip()) # Output: "hello"

8. rstrip()
Removes spaces from the **right** side of the string.
Syntax:variablename.rstrip()
Example:
s = "hello   "
print(s.rstrip()) # Output: "hello"

10. replace()
Replaces old substring with a new one. Doesn’t change the original string unless reassigned.
Syntax:variablename.replace('old data','new data')
Example:
s = "hello world"
print(s.replace("world", "Python")) # Output: "hello Python"

11. count()
Counts how many times a substring occurs in the string.
Syntax:variablename.count('character or data or string')
Example:
s = "banana"
print(s.count("a")) # Output: 3

12. startswith()
Returns True if the string begins with the specified value.
Syntax:variablename.startswith('string or character')
Example:
s = "python"
print(s.startswith("py")) # Output: True

13. endswith()

Checks if the string ends with a given substring. Returns True or False.
Syntax:variablename.endswith('string or character')
Example:
```
s = "python"
print(s.endswith("on"))  # Output: True
```

14. isalpha()
Returns True if all characters are alphabetic (no digits or symbols).
Syntax:variablename.isalpha()
Example:
```
s = "hello"
print(s.isalpha())  # Output: True
```

15. isdigit()
Checks if all characters in the string are digits (0-9).
Syntax:variablename.isdigit()
Example:
```
s = "12345"
print(s.isdigit())  # Output: True
```

16. isalnum()
Returns True if the string has only letters and numbers. No spaces or symbols allowed.
Syntax:variablename.isalnum()
Example:
```
s = "abc123"
print(s.isalnum())  # Output: True
```

17. islower()
Returns True if all characters are lowercase.
Syntax:variablename.islower()
Example:
```
s = "hello"
print(s.islower())  # Output: True
```

18. isupper()
Returns True if all characters are uppercase.
Syntax:variablename.isupper()
Example:
```
s = "HELLO"
print(s.isupper())  # Output: True
```

19. isspace()
Returns True if the string only contains whitespace characters.
Syntax:variablename.isspace()
Example:
```
s = "   "
print(s.isspace())  # Output: True
```

20. split()
Splits a string into a list using space or a given separator.
Syntax:variablename.split()
Example:
```
s = "Python is easy"
print(s.split())  # Output: ['Python', 'is', 'easy']
```

21. join()
Combines a list of strings into a single string using a separator.

Syntax:variablename.join()

Example:
words = ['Python', 'is', 'fun']
print(" ".join(words)) # Output: "Python is fun"
EXample2:
 s = 'appa'
'a'.join(s) #Output:'aapapaa'

22. swapcase()
Changes uppercase letters to lowercase and vice versa.
Syntax:variablename.swapcase()
Example:
s = "PyThOn"
print(s.swapcase()) # Output: "pYtHoN"

3.Inbuilt function on List:

1. append()
Description:
Adds a single element to the **end** of the list.
Syntax:variablename.append(element)
Example:
fruits = ['apple', 'banana']
fruits.append('cherry')
print(fruits) #Output: ['apple', 'banana', 'cherry']

2. clear()
Description:
Removes **all elements** from the list.
Syntax:variablename.clear()
Example:
nums = [1, 2, 3]
nums.clear()
print(nums) #Output: []

3. copy()
Description:
Returns a **shallow copy** of the list (not linked to the original list).
Syntax:variablename.copy()
Example:
a = [10, 20]
b = a.copy()
b.append(30)
print(a) # Output: [10, 20]
print(b) # Output: [10, 20, 30]

4. count()
Description:
Returns the number of times an element appears in the list.
Syntax:variablename.count(element)
Example:
nums = [1, 2, 2, 3]
print(nums.count(2)) # Output: 2

5. index()
Description:

Returns the **index** of the first occurrence of the specified value.
Syntax:variablename.index(element)
Example:

```
letters = ['a', 'b', 'c']
print(letters.index('b'))  # Output: 1
```

6. insert()

Description:
Inserts an element at the **specified position**.
Syntax:variablename.insert(index, element)
Example:

```
items = ['a', 'b']
items.insert(1, 'x')
print(items)  # Output: ['a', 'x', 'b']
```

7. pop()

Description:
Removes and returns the element at the given index. If index is not given, removes the last item.
Syntax:variablename.pop([index])
Example:

```
nums = [10, 20, 30]
print(nums.pop())    # Output: 30
print(nums.pop(0))   # Output: 10
print(nums)          # Output: [20]
```

8. remove()

Description:
Removes the **first occurrence** of the specified value.
Syntax:variablename.remove(element)
Example:

```
colors = ['red', 'blue', 'green']
colors.remove('blue')
print(colors)  # Output: ['red', 'green']
```

9. reverse()

Description:
Reverses the order of list elements **in-place**.
Syntax:variablename.reverse()
Example:

```
nums = [1, 2, 3]
nums.reverse()
print(nums)  #Output:  [3, 2, 1]
```

10. sort()

Description:
Sorts the list in ascending order by default. Can sort in descending or with custom key.
Syntax:variablename.sort(key=None, reverse=False)
Example:

```
a = [3, 1, 2]
a.sort()
print(a)  # [1, 2, 3]
a.sort(reverse=True)
print(a)  # [3, 2, 1]
```

Inbuilt functions on tuples:

1. count()
Purpose: Returns the **number of times a value appears** in the tuple.
Syntax:variablename.count(value)
t = (10, 20, 10, 30, 10)
print(t.count(10)) # Output: 3

2. index()
Purpose: Returns the **first index** of the given element.
t = (10, 20, 30, 10)
print(t.index(10)) # Output: 0
If the element is not found, it raises a ValueError.

3. len()
Purpose: Returns the **length** (number of elements) in the tuple.
Syntax:len(variablename)
t = (10, 20, 30)
print(len(t)) # Output: 3

4. max()
Purpose: Returns the **maximum value** in the tuple (only works if all elements are comparable).
Syntax:max(variablename)
t = (10, 20, 5)
print(max(t)) # Output: 20

5. min()
Purpose: Returns the **minimum value** in the tuple.
Syntax:min(variablename)
t = (10, 20, 5)
print(min(t)) # Output: 5

6. sum()
Purpose: Returns the **sum of numeric elements** in the tuple.
Example 1: All integers

t = (10, 20, 30, 40, 50)
print(sum(t)) # Output: 150
Example 2: With floats

t = (10, 20, 30, 40, 50, 5.5, 2.5)
print(sum(t)) # Output: 158.0
Example 3: With complex numbers

t = (10, 20, 30, 40, 50, 5.5, 2.5, 10+7j)
print(sum(t)) # Output: (168+7j)
Example 4: With True (treated as 1)

t = (10, 20, 30, 40, 50, 5.5, 2.5, 10+7j, True)
print(sum(t)) # Output: (169+7j)
Example 5: With a string –

t = (10, 20, 30, 40, 50, 5.5, 2.5, 10+7j, True, 'hi')
print(sum(t)) #Output: TypeError: unsupported operand types
sum() fails if **non-numeric types (like strings)** are present.

Inbuilt Functions On Set:

1. add(element)

Adds an element to the set.
Syntax:variablename.add(value)
s = {1, 2}
s.add(3)
print(s) # Output: {1, 2, 3}

2. clear()
Removes all elements from the set.
Syntax:variablename.clear()
s = {1, 2, 3}
s.clear()
print(s) # Output: set()

3. copy()
Returns a shallow copy of the set.

s = {1, 2}
new_set = s.copy()
print(new_set) # Output: {1, 2}

4. difference(other_set)
Returns a new set with elements in the original set but not in other_set.
Syntax: variablename1. **difference(**variablename2)
a = {1, 2, 3}
b = {2, 3, 4}
print(a.difference(b)) # Output: {1}
Print(a) #Output:{1, 2, 3}

5. difference_update(other_set)
Removes elements from the original set that are also in other set.(the original set will get effected)
a = {1, 2, 3}
b = {2, 3}
a.difference_update(b)
print(a) # Output: {1}

6. intersection(other_set)
Returns a new set with elements common to both sets.
Syntax: variablename1. **intersection**(variablename2)
a = {1, 2, 3}
b = {2, 3, 4}
print(a.intersection(b)) # Output: {2, 3}

7. intersection_update(other_set)
Keeps only elements found in both sets in the original set.(it will effect original set)
Syntax: variablename1. **intersection_update**(variablename2)
a = {1, 2, 3}
b = {2, 3, 4}
a.intersection_update(b)
print(a) # Output: {2, 3}

8. isdisjoint(other_set)
Returns True if sets have no elements in common.
Syntax: variablename1.isdidjoint(variablename2)
a = {1, 2}
b = {3, 4}
print(a.isdisjoint(b)) # Output: True

9. issubset(other_set)

Returns True if all elements of current set are in other_set.
Syntax: variablename1.issubset(variablename2)
a = {1, 2}
b = {1, 2, 3}
print(a.issubset(b)) # Output: True

10. issuperset(other_set)
Returns True if current set has all elements of other_set.
Syntax: variablename1.issuperset(variablename2)
a = {1, 2, 3}
b = {1, 2}
print(a.issuperset(b)) # Output: True

11. pop()
Removes and returns a random element.
Syntax: variablename1.pop(v)
s = {1, 2, 3}
print(s.pop()) # Randomly removes an element (e.g., 1)
print(s) # Remaining elements
Raises KeyError if the set is empty.

12. remove(element)
Removes specified element. Raises error if not found.
Syntax: variablename1.remove(value)
s = {1, 2, 3}
s.remove(2)
print(s) # Output: {1, 3}
s.remove(5) # KeyError if uncommented

13. union(other_set)
Returns a new set with all elements from both sets.
Syntax: variablename1.union(variablename2)
a = {1, 2}
b = {2, 3}
print(a.union(b)) # Output: {1, 2, 3}

14. update(other_set)
Adds elements from another set to the current set.

Syntax: variablename1.update(variablename2)
a = {1, 2}
b = {3, 4}
a.update(b)
print(a) # Output: {1, 2, 3, 4}
print(d) # Output: {'a': 1, 'b': 2}

Inbuilt functions On Dictionary

d = {'a': 1, 'b': 2, 'c': 3}
1. clear()
Removes all items from the dictionary.
Syntax: variablename.clear()
d.clear()
print(d) # Output: {}

2. copy()
Returns a shallow copy of the dictionary.
d = {'a': 1, 'b': 2}
copy_d = d.copy()

```
print(copy_d)  # Output: {'a': 1, 'b': 2}
```

3. get(key))

Returns the value of the key.

Syntax: variablename.get('key')

```
d = {'a': 10,'b':20}
print(d.get('a'))      # Output: 10
```

4. items()

Returns a view object of (key, value) pairs.

Syntax: variablename.items()

```
d = {'a': 1, 'b': 2}
print(list(d.items()))  # Output: [('a', 1), ('b', 2)]
```

5. keys()

Returns a view object of dictionary keys.

Syntax: variablename.keys()

```
d = {'a': 1, 'b': 2}
print(list(d.keys()))   # Output: ['a', 'b']
```

6. pop(key)

Removes the specified key and returns the value.

Syntax: variablename.pop(key)

```
d = {'a': 1, 'b': 2}
print(d.pop('a'))       # Output: 1
print(d)                # Output: {'b': 2}
```

7. popitem()

Removes and returns the last inserted key-value pair.

Syntax: variablename.popitem()

```
d = {'a': 1, 'b': 2}
print(d.popitem())  # Output: ('b', 2)
print(d)            # Output: {'a': 1}
```

8. update([otherset]):

Updates the dictionary with elements from another dictionary or from an iterable of key-value pairs.

Syntax:variablename.update(dictionary)

```
d = {'a': 1}
d.update({'b': 2, 'c': 3})
print(d)  # Output: {'a': 1, 'b': 2, 'c': 3}
```

9.. values()

Returns a view object of all values in the dictionary.

Syntax: variablename.values()

```
d = {'a': 1,'b':2,'c':3}
print(list(d.values()))  # Output: [1, 2, 3]
```